

LAB # 6 Threads and Threads

Objectives

- To demonstrate the creation of threads using the POSIX threads library.
- To perform thread creation by dynamic memory allocation.
- To show synchronization among the threads using Binary Semaphore.

Pre-Lab Theory:

Thread Creation:

`#include <pthread.h>`

synopsis:

```
int pthread_create(pthread_t *tid, pthread_attr_t *  
attr, dprocess, void* arg)
```

Description:

tid : Thread id

attr_t: thread attribute object

dprocess: Thread routine

arg : The argument to be passed to the thread routine

return: negative value if thread not created.

Thread Routine:

Synopsis:

```
void* dprocess(void * arg)
```

Threads Joining:

Synopsis:

```
int pthread_join(pthread_t * tid, void* status)
```

Description:

The main thread (process) will send a join call to the specific thread ID

LAB # 6 Threads and Threads

Thread Id:

`pthread_t pthread_self()`

Description: *The thread can call this function to display its thread id.*

Threads Synchronization:

Thread synchronization is defined as a mechanism that ensures that two or more concurrent processes or threads do not simultaneously execute some particular program segment known as a critical section.

Binary Semaphore:

Binary semaphores are synchronization mechanisms that have integer values that range from 0 (zero) to 1 (one). As a result, this type of semaphore gives a single point of access to a key portion. It signifies that only one thread will have simultaneous access to the critical section.

Mutex:

A mutex is a locking mechanism used to synchronize access to a resource. Only one thread can acquire the mutex. It means there is ownership associated with a mutex, and only the owner can release the lock (mutex).

```
pthread_mutex_t mutex =  
PTHREAD_MUTEX_INITIALIZER; int  
pthread_mutex_lock(pthread_mutex_t * mutex);  
int pthread_mutex_unlock(pthread_mutex_t *  
mutex);
```

LAB # 6 Threads and Threads

In-Lab Tasks:

Task 1: *Create a thread with default attributes and the thread will display its thread id.*

steps:

1. *Define a doprocess() function above the main() function that will use pthread_self() function to print the thread id.*
2. *Call the pthread_create() function in main() to create a thread.*
3. *Call the pthread_join() function to send a join call to the newly created thread.*

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

// doprocess() function above the main() function that will use
// pthread_self() function to print the thread id

void* doprocess(void* arg) {
    printf("Thread ID: %lu\n", pthread_self()); //longunsigned_lu
    //return NULL;
}

int main() {
    pthread_t tid;
    int err;
    // Call the pthread_create() function in main() to create a
    thread
    err = pthread_create(&tid, NULL, doprocess, NULL);
    //=> (address of tid , thread attribute , function_pointer to
    that which a thread will run , args to that function)
    // pthread_create returns 0 on success, or an error number
    otherwise
    if(err!=0){
        fprintf(stderr, "pthread_create failed: %d\n", err); //
    fprintf
        //printf() expects a format string (type const char*) as
    its
        //first argument.
        //stderr is a FILE pointer, not a string.
        return 1;
    }
    // Call the pthread_join() function to send a join call to the
```

LAB # 6 Threads and Threads

```
newly created
    // thread.
    err = pthread_join(tid, NULL);
//=> (threadid , return-value {where to store the return value of
the }
// what if we have to store it
// void* retval; // to capture returned value
//pthread_join(tid, &retval); // capture return value
//printf("Returned value = %d\n", *(int*)retval);
//free(retval); // cleanup
    if(err!=0){
        fprintf(stderr, "pthread_join failed: %d\n", err);
        return 1;
    }
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab6$ ./task1
Thread ID: 133398788044480
```

Task 2: Create 5 threads with default attributes and each thread will display its thread ID.

Steps:

1. *Declare an array of `pthread_t` `tid[5]`*
2. *Use the same `doprocess()` function in Task1*
3. *call `pthread_create()` function in the for loop*
4. *call `pthread_join()` to send a join call to the newly created threads*

Code:

```
#include <stdio.h>
#include <pthread.h>

void* doprocess(void* arg) {
    printf("Thread ID: %lu\n", pthread_self());
    return NULL;
}

int main() {
    pthread_t tid[5];
    for (int i = 0; i < 5; i++) {
        pthread_create(&tid[i], NULL, doprocess, NULL);
    }
}
```

LAB # 6 Threads and Threads

```
    for (int i = 0; i < 5; i++) {  
        pthread_join(tid[i], NULL);  
    }  
  
    return 0;  
}
```

Output:

muhammad-ahmad@latitude-7490:~/os/lab6\$./task2

Thread ID: 139305542481600

Thread ID: 139305525696192

Thread ID: 139305534088896

Thread ID: 139305517303488

Thread ID: 139305508910784

***Task 3:** Create n threads based on input entered by the user and allocate dynamic memory for n threads using `calloc()` function*

`pthread_t * tid;`

`tid = (pthread_t* )calloc(n,sizeof(pthread_t))`

Steps:

1. *prompt user in main() to enter the number of threads*
2. *In the main() function do the dynamic memory allocation for thread ids*
3. *Same steps as in the task*

Code:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
void* doprocess(void* arg) {  
    printf("Thread ID: %lu\n", pthread_self());  
    return NULL;  
}  
  
int main() {  
    // 1. prompt user in main() to enter the number of threads  
    int n;  
    printf("Enter number of threads: ");
```

LAB # 6 Threads and Threads

```
scanf("%d", &n);
// dynamically allocate space for tids wrt user count
pthread_t* tid;
tid = (pthread_t*)calloc(n, sizeof(pthread_t));

// 2. In the main() function do the dynamic memory
allocation for thread ids
for (int i = 0; i < n; i++) {
    pthread_create(&tid[i], NULL, doprocess, NULL);
}
for (int i = 0; i < n; i++) {
    pthread_join(tid[i], NULL);
}
//free the dynamically allocated space
free(tid);
return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab6$ ./task3
Enter number of threads: 2
Thread ID: 135076671125184
Thread ID: 135076662732480
```

Task 4: Display the thread sequence number as well before the thread id in task 3 by passing loop variable *i* to the thread routine(*doprocess*).

Steps:

In doprocess() do the necessary changes to typecast the argument

int i;

*i = *((int*)(arg))*

Modified doprocess() function:

```
void* doprocess(void* arg) {
    int i = *((int*)arg);
```

LAB # 6 Threads and Threads

```
    printf("Thread %d – ID: %lu\n", i, pthread_self());  
    return NULL;  
}
```

write pthread_create() call below:

```
pthread_create(&tid[i], NULL, doprocess, (void*)&i);
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab6$ ./task4
```

```
Enter number of threads: 3
```

```
Thread 1 – ID: 137256031483584
```

```
Thread 2 – ID: 137256023090880
```

```
Thread 3 – ID: 137256014698176
```

Highlight the problem in the output with the reason

→ the output looks correct, it is not guaranteed to be correct - every time you run it.

→ Sometimes, all threads may print the same sequence number

Task 5: Correct the problem in Task 4 by taking a global integer array for sequence numbers.

Task 5(a) Static allocation for integer sequence array for 5 threads and initialize it from 1 to 5

Steps:

- 1. Declaration and initialization of integer sequence array*
- 2. Pass the sequence number of thread from the integer sequence array with the index of for loop variable i to the doprocess() function.*

integer array declaration and initialization code

```
int seq[100]; // Global array for sequence numbers
```

Modified pthread_create() call

```
for (int i = 0; i < n; i++) {  
    seq[i] = i + 1;  
    pthread_create(&tid[i], NULL, doprocess, (void*)&seq[i]);  
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab6$ ./task5
```

LAB # 6 Threads and Threads

```
Enter number of threads (<=100): 4
Thread 1 – ID: 126913448244928
Thread 2 – ID: 126913439852224
Thread 3 – ID: 126913431459520
Thread 4 – ID: 126913423066816
```

Highlight the problem (if any) in the output

→ There is **no problem** in this output.

→ Each thread receives its own copy of `seq[i]`

Threads Synchronization:

Task 6: Introduce a global integer counter variable initialized to zero value. Each thread will increment the counter variable and print it with its thread id.

Steps:

Introduce a delay using a for loop with 1 billion iterations in the `doprocess()` function

```
for(int i=0;i<1000000000;i++);
```

Modified code of `doprocess()`

```
void* doprocess(void* arg) {
    for (int i = 0; i < 1000000000; i++); // Delay
    counter++;
    printf("Counter = %d – Thread ID: %lu\n", counter,
pthread_self());
    return NULL;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab6$ ./task6
Counter = 1 – Thread ID: 128868881004224
Counter = 2 – Thread ID: 128868855826112
Counter = 3 – Thread ID: 128868872611520
Counter = 4 – Thread ID: 128868864218816
Counter = 5 – Thread ID: 128868847433408
Final Counter: 5
```

Highlight the problem in the output and **Identify the critical section**

Increment (`counter++`) is not atomic – it involves three steps:

1. Read the counter value
2. Increment it
3. Write the new value back

If two threads perform these steps simultaneously, they can

LAB # 6 Threads and Threads

overwrite each other's updates → **race condition**.

Critical Section:

The **critical section** is the part of the code that accesses and modifies the shared resource:

```
counter++;  
printf("Counter = %d – Thread ID: %lu\n", counter, pthread_self());
```

Task 7: Modify the program in Task 6 to declare and initialize a global mutex(Binary Semaphore) variable to synchronize the thread for critical section in Task 6

Steps:

use mutex lock and unlock functions to protect the critical section and write output.

Modified doprocess() function

```
void* doprocess(void* arg) {  
    for (int i = 0; i < 1000000000; i++); // Delay  
  
    pthread_mutex_lock(&lock); // Enter critical  
    section ,apply lock  
    counter++;  
    printf("Counter = %d – Thread ID: %lu\n", counter,  
pthread_self());  
    pthread_mutex_unlock(&lock); // Exit critical  
    section ,apply unlock  
  
    return NULL;  
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab6$ ./task7  
Counter = 1 – Thread ID: 124636262028992  
Counter = 2 – Thread ID: 124636253636288  
Counter = 3 – Thread ID: 124636270421696  
Counter = 4 – Thread ID: 124636278814400  
Counter = 5 – Thread ID: 124636245243584  
Final Counter: 5
```